COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	9/3/26	**DATE SUBMITTED:**	9/3/26
NAME:	Cardenas, Lea Marie De Vera, Sophia Pangyarihan,Jenny Rose Peñaflorida,Wrio Carlo Prieto, Irish Crystle	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK 7.1 USING SORTING ALGORITHMS

1. PROCEDURE OUTPUT

Answer the following questions:

1. What is sorting algorithm?

2.Where can sorting algorithms be used?

3. Explain the different types of sorting algorithms.

4. Give sample programs in C++ that uses sorting algorithms, specifically selection sort, insertion sort, and bubble sort. Explain how the programs work.

1. What is a sorting algorithm?

Basically, a sorting algorithm is just a set of steps to arrange a list of data/stuff into an orderly manner, typically smallest to largest sorting, or the other way around. The user gives a messy or partially organized list, and the program code hands you back the same data, just rearranged so it actually follows a pattern, like each item being less than or equal to the one right after it.

2. Where can sorting algorithms be used?

Honestly, sorting pops up is basically everywhere once you start paying attention:

- **Searching — you can't really do binary search unless your data's sorted first, so sorting usually comes before it**
- **Databases — think indexing, ORDER BY queries, joining tables together more efficiently**
- **User interfaces — sorting your files by date, products by price, contacts alphabetically, stuff like that**

- **Data analysis — finding medians, percentiles, spotting duplicates, all of that usually needs sorted data first**
- **Scheduling systems — arranging tasks or processes by priority or deadline (kind of like that round-robin scheduler thing we did earlier with queues)**
- **Graphics and games — sorting objects by depth so things render in the right order**
- **Networking — stuff like routing tables and prioritizing packets**

In summary, sorting algorithms are useful especially on data that needs to be organized.

3. Different types of sorting algorithms

Based on how they compare things:

- **Bubble Sort —** keeps comparing the adjacent elements and swaps them if they're in the wrong place, so the biggest values slowly "bubble" their way to the end with each pass. Easy to understand, but not exactly fast ($O(n^2)$).
- **Selection Sort —** finds the smallest value first and puts it where it must be properly placed. Still $O(n^2)$, but at least it doesn't swap as much as bubble sort does.
- **Insertion Sort —** builds up the sorted list bit by bit, taking each new item and slotting it into the right spot among what's already sorted. The worst case is still $O(n^2)$, but it's actually really efficient if the data's already close to sorted.
- **Merge Sort —** splits the list in half over and over, sorts each half on its own, then merges everything back together. Runs at $O(n \log n)$ and keeps items in the same relative order (stable), but it does need extra memory to work.
- **Quick Sort —** picks a "pivot" value and splits the list into stuff smaller than it and stuff bigger than it, then keeps doing that recursively. Usually $O(n \log n)$ on average, but can slow down to $O(n^2)$ if you get unlucky with the pivot choices.
- **Heap Sort —** turns the list into a heap structure, then keeps pulling out the max value to build the sorted result. Always $O(n \log n)$, which is nice, but it's not stable.

Other ways to describe them:

- **Stable vs. Unstable —** stable sorts (like merge sort or insertion sort) keep equal elements in the same order they started in; unstable ones (like quick sort or selection sort) don't really care about that.
- **In-place vs. Out-of-Place —** in-place sorts (bubble, selection, insertion, quick) barely need any extra memory to run; out-of-place ones like merge sort need extra space that grows along with your input size.

Selection Sort — Real-World Program Example

Program: Leaderboard Ranking System for a Small Local Tournament

Imagine a badminton club runs a small tournament with, say, 20 players. After each round, the organizer needs to rank players by their scores to post standings on a bulletin board. A selection sort works well here because:

- The dataset is small (20 players), so its $O(n^2)$ time isn't a real drawback.
- It performs very few actual swaps — it scans to find the minimum (or maximum) score and moves it into place just once per pass. This matters when "swapping" means physically rearranging printed name tags

or database rows with heavier data attached, since fewer writes/swaps means less overhead.
- It's simple enough that a small in-house scoring app doesn't need a complex sorting library.

Insertion Sort — Real-World Program Example

Program: Live Order Queue for a Coffee Shop POS System

Picture a point-of-sale system at a coffee shop that keeps a running list of orders sorted by pickup time, and new orders keep arriving one at a time throughout the day. Insertion sort fits naturally because:

- The list is *already mostly sorted* at any given moment — a new order just needs to be slotted into its correct spot among existing ones. Insertion sort's inner loop barely does any work on nearly-sorted data, making it efficient here even though it's technically $O(n^2)$ in the worst case.
- It processes data incrementally, one new item at a time — exactly the pattern of how real orders come in — rather than needing the whole dataset upfront like some faster sorts.
- This is actually the same underlying idea used by "insert into sorted position" features in many real scheduling and calendar apps.

Bubble Sort — Real-World Program Example

Program: Beginner-Level Grade Sorting Tool for a Classroom Spreadsheet Macro

Consider a teacher's simple spreadsheet macro/script that sorts a class's 30 exam scores from lowest to highest so the teacher can quickly eyeball the grade distribution. Bubble sort is a reasonable (if not optimal) choice here because:

- The dataset is tiny (a single class of students), so performance differences between sorting algorithms are effectively unnoticeable to the user.
- It's the easiest sorting logic to write, read, and debug — ideal for a lightweight teaching tool, training script, or educational demo where code clarity matters more than raw speed.
- With the "early exit if no swaps happened" optimization, it also handles the common case of "the list only needs a couple of tweaks" (e.g., updating one late-arriving grade) quite efficiently, since it can finish in a single extra pass.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.